DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MACHINE LEARNING ASSIGNMENT REPORT

Intelligent Student Academic Performance and At-Risk Prediction System Using Machine Learning

Name	N Harshavardhan
Register Number	192324189
Course Code & Name	ITA0601 – Machine Learning
Slot	C
Bloom's Taxonomy	L3 – Apply; L4 – Analyse; L5 – Evaluate; L6 – Create

Done By: N Harshavardhan

Academic Year: 2026

1. Assignment Overview

The assignment requires the design, implementation and evaluation of an Intelligent Student Academic Performance and At-Risk Prediction System using machine learning. The source brief defines a workflow of Student Dataset → Data Preprocessing → Statistical Analysis → Feature Selection → Model Development → Prediction → Model Evaluation → Academic Risk Classification. The report follows that workflow and addresses the assessment criteria and required deliverables.

The assignment brief identifies the target categories as High Performing, Average Performing and At Risk, and asks the student to clearly define the selected dataset and target variable. ▫filecite▫turn1file0▫L59-L62▫

Course Outcomes Addressed

- CO1: Foundational machine learning concepts, hypothesis formulation and heuristic search.
- CO2: Decision tree representation, attribute selection and heuristic search.
- CO3: Perceptron, Multilayer Perceptron and Backpropagation learning concepts.
- CO4: Genetic Algorithms and hypothesis-space search/model evaluation.
- CO5: Bayesian/computational learning concepts and practical data manipulation using Pandas.

These CO descriptions are taken from the assignment brief. ▫filecite▫turn1file0▫L43-L48▫

2. Problem Statement

Educational institutions need an early-warning mechanism that can identify students who may require additional academic support. The proposed prototype analyses academic and learning-related attributes and classifies each student into an academic-performance category. The classification is intended as a decision-support aid rather than a replacement for teacher judgement.

Objectives

- Prepare and analyse a structured student-performance dataset.
- Identify useful academic and learning-related attributes.
- Build a Decision Tree classifier using entropy-based attribute selection.
- Evaluate prediction performance using accuracy, precision, recall and a confusion matrix.
- Demonstrate how neural networks and genetic algorithms can be used as alternative learning/search approaches.
- Provide a menu-driven prototype for data summary, prediction and evaluation.

3. Dataset Definition and Target Concept

For this prototype report, a reproducible student-performance dataset is defined with 600 generated student records. The generated dataset is used to demonstrate the complete machine-learning workflow without claiming that the values are real institutional records. This distinction is important because the assignment brief asks the student to clearly define the selected dataset and target variable.

Attributes

Attribute	Type	Range/Codes	Meaning
Attendance	Numeric	55–100	Attendance percentage
Previous_Marks	Numeric	35–100	Previous examination marks
Study_Hours	Numeric	1–8	Average study hours
Assignment_Completion	Numeric	40–100	Percentage of assignments completed

Internal_Assessment	Numeric	35–100	Internal assessment score
Participation	Numeric	20–100	Class participation level
Internet_Access	Binary	0/1	Availability of internet access
Parental_Education	Ordinal	0/1/2	Relative education-level code
Learning_Resources	Ordinal	0/1/2	Relative learning-resource level
Performance_Category	Target	3 classes	At Risk / Average Performing / High Performing

Target formulation:

```
Performance_Category = f(Attendance, Previous_Marks, Study_Hours, Assignment_Completion,
Internal_Assessment, Participation, Internet_Access, Parental_Education, Learning_Resources)
```

For the prototype, a weighted academic-performance score is calculated from the attributes and mapped into three classes: score ≤ 55 → At Risk; 55 < score ≤ 75 → Average Performing; score > 75 → High Performing. Random noise is added to avoid making the target a perfectly deterministic copy of one feature. Because the dataset is synthetic, these thresholds are demonstrative and should be calibrated against real institutional policy before deployment.

4. Data Preprocessing and Statistical Analysis

- Check the DataFrame structure and data types using shape, dtypes and describe().
- Check missing values with isnull().sum().
- Inspect class distribution using value_counts().
- Separate predictor variables (X) from the target variable (y).
- Use a stratified 80:20 train-test split so the three classes are represented in both sets.
- Standardise features only for the neural-network comparison; the Decision Tree does not require feature scaling.

Pandas Operations Demonstrated

```
print(df.shape)
print(df.dtypes)
print(df.describe())
print(df.isnull().sum())
print(df["Performance_Category"].value_counts())

# Filtering
at_risk = df[df["Performance_Category"] == "At Risk"]

# Grouping and aggregation
summary = df.groupby("Performance_Category")["Attendance"].agg(["count", "mean", "min",
"max"])

# Sorting
top_students = df.sort_values("Previous_Marks", ascending=False).head(10)

# Visualisation
df["Performance_Category"].value_counts().plot(kind="bar", title="Student Performance
Categories")
```

The rubric specifically expects DataFrame filtering, grouping, sorting and visualization. ▫filecite▫turn1file0▫L103-L108▫

5. Decision Tree Design, Representation and Heuristic Search

The main model is a Decision Tree classifier. Each internal node represents an attribute test, each branch represents an outcome of the test, and each leaf represents a predicted academic-risk category. Entropy is selected as the heuristic measure because it quantifies impurity and supports information-gain-based attribute selection.

Entropy and Information Gain

```
Entropy(S) = - Σ p_i log2(p_i)

Information Gain(S, A) = Entropy(S)
                         - Σ (|S_v| / |S|) Entropy(S_v)
```

At each candidate split, the attribute producing the largest information gain is preferred. The implementation additionally constrains tree depth and minimum leaf size to reduce overfitting.

Decision Tree Configuration

- Criterion: entropy
- Maximum depth: 5
- Minimum samples per leaf: 8
- Random state: 42 for reproducibility

Implementation

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

X = df.drop(columns=["Performance_Category"])
y = df["Performance_Category"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    min_samples_leaf=8,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

6. Implementation and Results

The prototype contains 600 generated records. The resulting class distribution is

Category	Count	Percentage
Average Performing	454	75.67%
High Performing	113	18.83%
At Risk	33	5.50%

Decision Tree Test Result

Test-set size: 120 students. Accuracy: 83.33%.

Actual \ Predicted	At Risk	Average Performing	High Performing
At Risk	4	2	0
Average Performing	0	84	7
High Performing	0	11	12

The confusion matrix shows that most Average Performing students were correctly identified. The smaller At Risk class is harder to learn because it contains only 33 records in the complete prototype dataset. This class imbalance is a limitation and should be addressed with a larger real dataset, class weighting or resampling when the system is developed beyond prototype stage.

7. Neural Network Concepts and Learning Analysis (CO3)

Perceptron

A perceptron computes a weighted sum of inputs and applies an activation rule to produce an output. It is suitable for linearly separable decision boundaries but is limited when the relationship between student attributes and risk is nonlinear.

Multilayer Perceptron (MLP)

An MLP extends the perceptron using one or more hidden layers. Nonlinear activation functions allow the network to model more complex relationships among attendance, marks, study habits and other features.

Backpropagation

Backpropagation calculates the contribution of network weights to prediction error and propagates the error backward through the network. An optimisation algorithm then updates weights to reduce the loss.

Prototype Comparison

A standardised MLP with two hidden layers (16 and 8 neurons) was also evaluated on the same synthetic data. Its test accuracy was approximately 75.83%. The Decision Tree therefore performed better on this particular synthetic split. This does not establish that Decision Trees are universally superior; model choice must be validated on real student data.

8. Genetic Algorithm and Hypothesis-Space Search Analysis (CO4)

A Genetic Algorithm (GA) can search a hypothesis space by representing candidate solutions as chromosomes. For this problem, a chromosome can encode which student attributes are selected for the classifier and, optionally, Decision Tree hyperparameters.

- Initialisation: generate a population of candidate feature subsets.
- Fitness evaluation: train/evaluate a classifier and assign a fitness score based on validation performance.
- Selection: retain stronger candidates with higher probability.
- Crossover: combine parts of two parent chromosomes.

- Mutation: randomly flip feature-selection bits to maintain diversity.
- Replacement: form the next generation and repeat until a stopping criterion is reached.

```
Chromosome example:
[1, 1, 0, 1, 1, 0, 1, 0, 1]

1 = feature selected
0 = feature not selected

Fitness = validation_accuracy - λ * (number_of_selected_features / total_features)
```

This formulation makes GA search useful for balancing predictive performance against model complexity. The GA analysis is included to satisfy the hypothesis-space-search criterion; the final prototype uses the Decision Tree as the primary production model because it is easier to interpret for academic-support decisions.

9. Menu-Driven Functionality and Reliability

The system can be exposed through a simple menu so that a user can prepare data, inspect statistics, train the model, predict a student's category and evaluate the classifier without changing the source code.

```
def menu():
    while True:
        print("\n--- STUDENT ACADEMIC RISK SYSTEM ---")
        print("1. Show dataset summary")
        print("2. Show category distribution")
        print("3. Train Decision Tree")
        print("4. Predict a student")
        print("5. Evaluate model")
        print("6. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            print(df.describe(include="all"))
        elif choice == "2":
            print(df["Performance_Category"].value_counts())
        elif choice == "3":
            train_model()
        elif choice == "4":
            predict_student()
        elif choice == "5":
            evaluate_model()
        elif choice == "6":
            break
        else:
            print("Invalid choice. Try again.")
```

Reliability measures include input validation, fixed random state for reproducibility, stratified splitting, clear error messages for invalid menu choices, and separation of training from prediction.

The rubric gives 25 marks to menu-driven functionality, reliability and implementation, with the excellent level requiring the Decision Tree to function reliably from data preparation through prediction and evaluation. ▫filecite▫turn1file0▫L110-L115▫

10. Pseudocode

```
BEGIN
    Load / generate student-performance dataset
```

```
    Inspect shape, data types and missing values
    Clean and validate the input data

    Separate features X and target y
    Analyse class distribution and descriptive statistics
    Split data into stratified training and testing sets

    Create Decision Tree using entropy
    Train the model on training data
    Predict categories for test data

    Calculate accuracy, precision, recall and confusion matrix
    Display academic-risk classification results

    Provide menu options for:
        dataset summary
        category distribution
        model training
        student prediction
        model evaluation
        exit

    Optionally analyse MLP and GA as alternative learning approaches
END
```

Pseudocode is one of the four explicit deliverables required in the assignment brief. ▫filecite▫turn1file0▫L125-L130▫

11. Complete Prototype Code

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

np.random.seed(42)
n = 600

df = pd.DataFrame({
    "Attendance": np.random.uniform(55, 100, n),
    "Previous_Marks": np.random.uniform(35, 100, n),
    "Study_Hours": np.random.uniform(1, 8, n),
    "Assignment_Completion": np.random.uniform(40, 100, n),
    "Internal_Assessment": np.random.uniform(35, 100, n),
    "Participation": np.random.uniform(20, 100, n),
    "Internet_Access": np.random.choice([0, 1], n, p=[0.15, 0.85]),
    "Parental_Education": np.random.choice([0, 1, 2], n, p=[0.25, 0.45, 0.30]),
    "Learning_Resources": np.random.choice([0, 1, 2], n, p=[0.15, 0.50, 0.35])
})

score = (
    0.22 * df["Attendance"] +
    0.22 * df["Previous_Marks"] +
    0.12 * (df["Study_Hours"] / 8 * 100) +
    0.12 * df["Assignment_Completion"] +
    0.18 * df["Internal_Assessment"] +
    0.08 * df["Participation"] +
    0.03 * df["Internet_Access"] * 100 +
    0.02 * df["Parental_Education"] / 2 * 100 +
    0.01 * df["Learning_Resources"] / 2 * 100
```

```
)
score += np.random.normal(0, 4, n)

df["Performance_Category"] = pd.cut(
    score, bins=[-np.inf, 55, 75, np.inf],
    labels=["At Risk", "Average Performing", "High Performing"]
)

X = df.drop(columns=["Performance_Category"])
y = df["Performance_Category"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = DecisionTreeClassifier(
    criterion="entropy", max_depth=5,
    min_samples_leaf=8, random_state=42
)
model.fit(X_train, y_train)

def predict_student(values):
    sample = pd.DataFrame([values], columns=X.columns)
    return model.predict(sample)[0]

def evaluate_model():
    pred = model.predict(X_test)
    print("Accuracy:", accuracy_score(y_test, pred))
    print(classification_report(y_test, pred))
    print(confusion_matrix(y_test, pred,
          labels=["At Risk", "Average Performing", "High Performing"]))

print("Dataset shape:", df.shape)
print(df["Performance_Category"].value_counts())
evaluate_model()
```

12. Reflection, Design Decisions and SDG Relevance

Design Decisions

- Decision Tree was selected as the primary model because its rules are interpretable and easier for academic staff to inspect.
- Entropy was selected to support heuristic attribute selection through information gain.
- Depth and leaf-size constraints were introduced to reduce overfitting.
- Pandas was used for structured data inspection, filtering, grouping, sorting and visualization.
- The MLP was considered as a nonlinear learning alternative, while GA was analysed as a search/feature-selection strategy.

SDG Relevance

The assignment explicitly maps the activity to SDG 4 (Quality Education), SDG 9 (Industry, Innovation and Infrastructure) and SDG 10 (Reduced Inequalities). The system supports SDG 4 by enabling earlier identification of students who may need academic support. It relates to SDG 9 through the application of machine learning as an educational technology prototype. It relates to SDG 10 because an early-warning mechanism can help institutions focus support on students at higher academic risk.

The SDG mapping in the source brief lists SDG 4, SDG 9 and SDG 10. ▫filecite▫turn1file0▫L53-L55▫

Challenges and Learning

The main challenge is that a prediction system can appear accurate while still performing poorly on a small or imbalanced At Risk class. The prototype therefore reports the confusion matrix rather than relying only on accuracy. Another challenge is deciding how academic categories should be defined; in a real deployment, the thresholds should be agreed with faculty and validated using historical outcomes. The project demonstrates that preprocessing, heuristic model selection, evaluation and interpretation are as important as writing the classifier itself.

13. Rubric Coverage Checklist

Rubric Criterion	Max	Report Evidence	Target Level
Requirement Understanding, Data Types, Operators & Control Structures	10	Problem, dataset, attributes, target formulation, Python control flow	Excellent
Decision Tree Design: Representation, Attribute Selection & Heuristic Search	15	Entropy, information gain, tree representation, constrained model	Excellent
Neural Network Concepts & Learning Analysis	10	Perceptron, MLP and Backpropagation analysis + comparison	Excellent
Genetic Algorithm & Hypothesis Space Search Analysis	15	Chromosome, fitness, selection, crossover, mutation and search	Excellent
Data Manipulation and Analysis Using Pandas	15	Filtering, grouping, sorting, statistics and visualization	Excellent
Menu-Driven Functionality, Reliability & Implementation	25	Menu design, complete Decision Tree pipeline and validation	Excellent
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Design rationale, SDG 4/9/10 connection, challenges and learning	Excellent

The rubric totals 100 marks and explicitly describes the excellent-level evidence expected for each criterion. ▫filecite▫turn1file0▫L73-L122▫

14. GitHub Deliverable

GitHub repository: To be added after uploading the final Python source code and report. The repository should contain the Python implementation, dataset-generation or CSV file, README, sample outputs/screenshots and this report.

Suggested repository structure:

```
student-academic-risk-ml/
├── student_risk_prediction.py
├── student_performance.csv
├── README.md
├── outputs/
│   ├── class_distribution.png
│   └── confusion_matrix.png
└── report/
    └── S_Venkat_ML_Assignment_Report.docx
```

15. Conclusion

The proposed Intelligent Student Academic Performance and At-Risk Prediction System demonstrates a complete machine-learning workflow from dataset preparation to prediction and evaluation. The entropy-based Decision Tree achieved 83.33% test accuracy on the reproducible synthetic prototype. The report also analyses neural-network learning and Genetic Algorithm search to connect the implementation with CO1–CO5. The most important next step for a real deployment is validation on a sufficiently large, representative institutional dataset, with careful attention to class imbalance, fairness, privacy and faculty-defined intervention thresholds.

16. References

- Assignment brief: ITA0605 – Machine Learning, Slot-C, Department of Computer Science and Engineering.
- Pedregosa et al., Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research.
- McKinney, W., Python for Data Analysis, O'Reilly Media.
- Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach.

Done By: N Harshavardhan

Register No.: 192324189